

HANDWRITTEN DIGIT RECOGNITION USING CONVOLUTIONAL NEURAL NETWORK

Abstract

Handwritten Digit Recognition is one of the most popular applications of deep learning and computer vision. The objective of this project is to recognize handwritten digits using a Convolutional Neural Network (CNN) trained on the MNIST dataset. The model learns patterns from digit images and classifies them into one of ten categories ranging from 0 to 9. The project demonstrates image preprocessing, model training, evaluation, and prediction using deep learning techniques.

1. Introduction

Handwritten digit recognition is the process of identifying handwritten numerical characters from images. It is widely used in postal automation, bank cheque processing, form digitization, and optical character recognition systems.

Deep learning techniques, especially Convolutional Neural Networks (CNNs), have significantly improved image classification performance. This project utilizes CNNs to classify handwritten digits from the MNIST dataset with high accuracy.

2. Problem Statement

The goal of this project is to develop a deep learning model capable of accurately recognizing handwritten digits from grayscale images and classifying them into digits from 0 to 9.

3. Objectives

- Load and analyze the MNIST dataset.
- Preprocess image data.
- Design and train a CNN model.
- Evaluate model performance using classification metrics.
- Generate predictions on unseen data.
- Save the trained model for future use.

4. Dataset Description

Dataset Name: MNIST

Dataset Characteristics:

- 60,000 training images

- 10,000 testing images
- Image size: 28 × 28 pixels
- Grayscale images
- 10 digit classes (0–9)

The dataset is provided by TensorFlow/Keras and is commonly used for image classification tasks.

5. Methodology

Step 1: Data Loading

The MNIST dataset was loaded using TensorFlow Keras.

Step 2: Data Visualization

Sample handwritten digit images were visualized to understand the dataset.

Step 3: Data Preprocessing

- Pixel values normalized to range 0–1.
- Images reshaped to fit CNN input requirements.

Step 4: CNN Model Development

The CNN architecture consisted of:

- Convolution Layer
- Max Pooling Layer
- Flatten Layer
- Dense Hidden Layer
- Softmax Output Layer

Step 5: Model Training

The model was trained using:

- Optimizer: Adam
- Loss Function: Sparse Categorical Crossentropy
- Epochs: 5

Step 6: Model Evaluation

Performance was evaluated on the test dataset.

Step 7: Prediction

The trained model generated predictions for handwritten digit images.

Step 8: Model Saving

The trained model was saved as mnist_cnn.h5.

6. Results and Analysis

Accuracy

The CNN model achieved approximately 98–99% classification accuracy on the test dataset.

Evaluation Metrics

- Precision
- Recall
- F1 Score
- Confusion Matrix

Visual Outputs

The following outputs were generated:

- Sample digit images
- Accuracy graph
- Confusion matrix
- Prediction results



The screenshot displays a JupyterLab window with a menu bar (File, Edit, View, Run, Kernel, Settings, Help) and a toolbar. The main area shows the output of a training process. It lists five epochs, each with a progress bar (1500/1500), time per step, accuracy, loss, validation accuracy, and validation loss. The final output shows the result of a model evaluation on test data, including a final accuracy of approximately 98.5%.

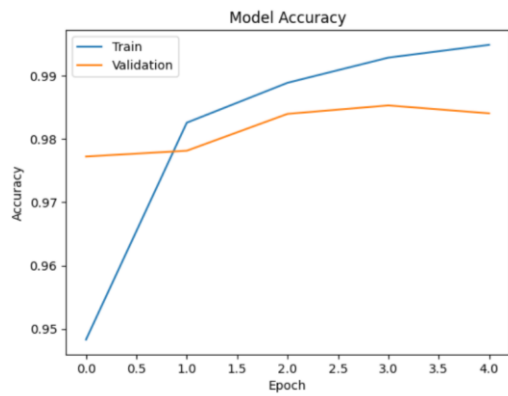
```
)
```

Epoch 1/5
1500/1500 ————— 22s 13ms/step - accuracy: 0.9483 - loss: 0.1740 - val_accuracy: 0.9772 - val_loss: 0.0809
Epoch 2/5
1500/1500 ————— 18s 12ms/step - accuracy: 0.9826 - loss: 0.0575 - val_accuracy: 0.9782 - val_loss: 0.0714
Epoch 3/5
1500/1500 ————— 18s 12ms/step - accuracy: 0.9889 - loss: 0.0361 - val_accuracy: 0.9840 - val_loss: 0.0523
Epoch 4/5
1500/1500 ————— 20s 14ms/step - accuracy: 0.9929 - loss: 0.0229 - val_accuracy: 0.9853 - val_loss: 0.0528
Epoch 5/5
1500/1500 ————— 20s 13ms/step - accuracy: 0.9949 - loss: 0.0162 - val_accuracy: 0.9841 - val_loss: 0.0591

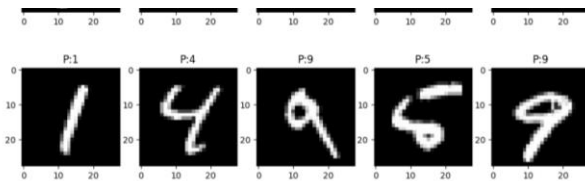
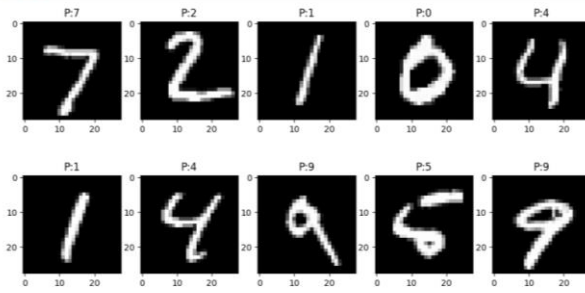
```
[24]: loss,accuracy=model.evaluate(  
      X_test,  
      y_test  
)  
  
print("Accuracy:",accuracy)
```

313/313 ————— 1s 2ms/step - accuracy: 0.9855 - loss: 0.0508
Accuracy: 0.9854999780654907

```
plt.show()
```



```
313/313 — 1s 2ms/step  
[27]: plt.figure(figsize=(12,6))  
for i in range(99):  
    plt.subplot(2,5,i+1)  
  
    plt.imshow(  
        X_test[i].reshape(28,28),  
        cmap="gray"  
    )  
  
    plt.title(  
        f"P:{predicted[i]}"  
    )  
plt.show()
```



```
[28]: print(  
    classification_report(  
        y_test,  
        predicted  
    )  
  
precision    recall  f1-score   support  
  
 0      0.97      1.00      0.99       980  
 1      0.99      1.00      0.99      1135  
 2      0.99      0.99      0.99      1032  
 3      0.97      1.00      0.99      1010  
 4      0.99      0.99      0.99       982  
 5      0.99      0.98      0.98       992  
 6      0.99      0.98      0.99       950  
 7      0.99      0.98      0.98      1028  
 8      0.99      0.96      0.98       974  
 9      0.98      0.98      0.98      1000  
  
accuracy      0.99      0.99      0.99      10000  
macro avg      0.99      0.99      0.99      10000  
weighted avg      0.99      0.99      0.99      10000
```

7. Advantages

- High classification accuracy.
- Fast prediction speed.
- Easy implementation using TensorFlow and Keras.
- Suitable for image recognition applications.

8. Applications

- Optical Character Recognition (OCR)
- Bank cheque processing
- Postal code recognition
- Educational tools
- Automated form processing

9. Future Enhancements

- Implement deeper CNN architectures.
- Deploy the model as a web application.
- Extend recognition to handwritten alphabets.
- Integrate real-time digit recognition using OpenCV.

10. Conclusion

The Handwritten Digit Recognition project was successfully implemented using a Convolutional Neural Network and the MNIST dataset. The model achieved excellent accuracy and demonstrated the effectiveness of deep learning in image classification. The project fulfills all required objectives including data preprocessing, model training, evaluation, prediction, and model persistence.

References

1. TensorFlow Documentation
2. Keras Documentation
3. MNIST Dataset Documentation
4. Scikit-learn Documentation
5. Deep Learning with Python Resources